Lab Worksheet: Proxy & Bridge Patterns in Java

Hands-On Practice Exercises • Real-World Structural Patterns

Student Name:	Student ID:	Date:	Score / Grade:
_____	_____	_____	_____

Lab Instructions: Complete all 6 tasks directly in your Java development environment. Test each scenario in `main()` to verify your output matches the expected logs. No external libraries are required—use plain Java classes and interfaces.

PART 1: THE PROXY PATTERN (Controlling & Mediating Access)

Task 1: The Bouncer (Access Control / Protection Proxy)

TIME: 3 MINS

Goal: *Prove that a proxy can act as a security guard to block unauthorized requests before the expensive object is ever touched.*

Requirements:

1. Open your `ProxyImage` class and add a `String userRole` field to its constructor:
   `public ProxyImage(String fileName, String userRole)`
2. Inside `show()`, check the caller's role before doing anything else:
   - If `userRole` is **not** `"ADMIN"`, print:
     `[ACCESS DENIED] You lack permission to view: <fileName>` and `return` immediately.
   - If `userRole` **is** `"ADMIN"`, proceed with lazy-loading and call `realImage.show()`.
3. In `main()`, instantiate two proxies: one for `"GUEST"` and one for `"ADMIN"`. Call `show()` on both.

Expected Console Output:

```
[ACCESS DENIED] You lack permission to view: salary_sheet.png
--> [SLOW] Reading salary_sheet.png from hard drive...
Displaying salary_sheet.png on screen!
```

Task 2: The View Counter (Tracking & Logging Proxy)

TIME: 5 MINS

Goal: *Learn how a proxy transparently tracks usage metrics without altering the core playback logic.*

Requirements:

1. Open your `ProxyVideo` class and add a private counter: `private int viewCount = 0;`
2. Inside `play()`:
   - Increment `viewCount` by 1.
   - Print an audit log: `--> [AUDIT] Video has been played <viewCount> time(s).`
   - Delegate to `realVideo.play()` as normal (lazy-loading on first view).
3. In `main()`, call `play()` on the same video three times in a row.

Expected Console Output:

```
--> [AUDIT] Video has been played 1 time(s).
--> [DOWNLOADING] Connecting to server & buffering 4K video: Cat_Memes.mp4
Playing video: Cat_Memes.mp4

--> [AUDIT] Video has been played 2 time(s).
Playing video: Cat_Memes.mp4

--> [AUDIT] Video has been played 3 time(s).
Playing video: Cat_Memes.mp4
```

Task 3: Mini-Challenge: School Wi-Fi Firewall

TIME: 10–15 MINS

Goal: *Build a complete proxy solution from scratch without copying previous class names.*

Requirements:

1. Create interface `Internet` with a single method: `void connectTo(String site);`
2. Create class `RealInternet implements Internet`:
   - `connectTo(String site)` prints: `Connected to site: <site>`
3. Create class `SchoolProxyInternet implements Internet`:
   - Contains a private reference: `RealInternet realInternet;`
   - Inside `connectTo(String site)`:
     - If `site` equals `"gaming.com"` or `"socialmedia.com"`, print: `[BLOCKED BY FIREWALL] Cannot access: <site>`
     - Otherwise, instantiate `realInternet` if `null`, and call `realInternet.connectTo(site)`.
4. In `main()`, create a `SchoolProxyInternet` instance and test connections to `"wikipedia.org"` and `"gaming.com"`.

Expected Console Output:

```
Connected to site: wikipedia.org
[BLOCKED BY FIREWALL] Cannot access: gaming.com
```

PART 2: THE BRIDGE PATTERN (Decoupling Abstraction from Implementation)

Task 4: The New Machine (Adding a Smart Light)

TIME: 3 MINS

Goal: *Prove that adding a new device requires touching ZERO lines of remote controller code.*

Requirements:

1. Create a new class `SmartLight implements Device`:
   - `turnOn()` prints: `Smart Light glows bright white.`
   - `turnOff()` prints: `Smart Light shuts off.`
   - `setVolume(int level)` prints: `Smart Light brightness set to: <level>%`
2. In `main()`:
   - Instantiate `Device light = new SmartLight();`
   - Pair it with an `AdvancedRemote remote = new AdvancedRemote(light);`
   - Call `remote.powerOn();` and `remote.mute();`

Expected Console Output:

```
Smart Light glows bright white.
[Remote] Mute button pressed!
Smart Light brightness set to: 0%
```

Task 5: The New Button (Adding a Party Remote)

TIME: 5 MINS

Goal: *Prove that adding a new controller feature requires touching ZERO lines of machine code.*

Requirements:

1. Create a class `PartyRemote extends BasicRemote`:
   - Constructor: `public PartyRemote(Device device) { super(device); }`
   - Add method `public void partyMode()`:
     - Prints: `[Party Remote] Pumping up the party!`
     - Calls `device.turnOn();` and cranks volume: `device.setVolume(100);`
2. In `main()`, instantiate a `Radio`, hand it to a `PartyRemote`, and invoke `partyMode()`.

Expected Console Output:

```
[Party Remote] Pumping up the party!
Radio starts playing static/music.
Radio volume knob turned to: 100
```

Task 6: Mini-Challenge: RPG Game Weapon System

TIME: 10–15 MINS

Goal: *Build a complete Bridge system from scratch to stop class multiplication ($M imes N$).*

Requirements:

1. **Implementor Hierarchy (Weapons):**
   - Interface `Weapon` with method `void use();`
   - Class `Sword implements Weapon` → `use()` prints: `Slash! Dealing 20 physical damage.`
   - Class `MagicWand implements Weapon` → `use()` prints: `Zap! Dealing 35 magic damage.`
2. **Abstraction Hierarchy (Characters):**
   - Abstract class `Hero` with a bridge slot: `protected Weapon weapon;` and a constructor that accepts `Weapon`.
   - Class `Knight extends Hero` with method `public void attack() { weapon.use(); }`
   - Class `Wizard extends Hero` with method `public void attack() { weapon.use(); }`
3. In `main()`:
   - Create a `Knight` equipped with a `Sword` and call `attack()`.
   - Create a `Knight` equipped with a `MagicWand` and call `attack()`.

Expected Console Output:

```
Slash! Dealing 20 physical damage.
Zap! Dealing 35 magic damage.
```